



COLLEGE CODE : 8203
COLLEGE NAME : AVC COLLEGE OF ENGINEERING
DEPARTMENT : INFORMATION TECHNOLOGY
STUDENT NM-ID : CA86DF453063BF99FCC40A1FFDE7E78F
ROLL NO : 23IT75
DATE : 08.09.2025

Completed the project named as Phase3
TECHNOLOGY PROJECTNAME: Weather Dashboard

SUBMITTED BY,

NAME: PRADEEP KUMAR R

MOBILE NO: 6379897924

Project Setup – Weather Dashboard

MVP Implementation

Project Setup:

The first step is to establish the foundation and connect all the required services.

Step	Description	Tools Involved
1. Project Initialization & Dependencies	Create the main project folders (backend, frontend) and initialize the Node.js backend. Install essential packages for the server, external API calls, and caching.	npm init -y, npm install express axios dotenv redis cors
2. Environment Configuration	Create a .env file in the backend directory to securely store critical keys and URLs: OPENWEATHER_API_KEY and REDIS_URL.	dotenv , Open Weather API
3. Server Initialization	Create backend/server.js to set up the Express.js application, defining the server port (PORT=5000) and configuring middleware.	Node.js, Express.js
4. Redis Connection Setup	Implement the connection logic for Redis within the server, establishing the client connection using the REDIS_URL from the environment file.	redis
5. Frontend File Structure	Create the core UI files in the frontend folder: index.html (structure), style.css (design), and app.js (client-side API calls to the backend).	HTML, CSS, JavaScript

Data Storage [local state & Database]

Weather Dashboard project uses a hybrid approach, relying on an External API as the source of truth and a Caching Database (Redis) for performance optimization, instead of a traditional relational or NoSQL database for long-term persistence.

1. Primary Data Source (Source of Truth)

The project's primary data storage is managed externally.

- Tool: Open Weather API
- Role: Acts as the Source of Truth for all real-time weather information.
- Data Flow: The backend calls the Open Weather API via Axios to fetch the current weather conditions (e.g., temperature, humidity, wind speed).

2. Database (Caching Layer)

The project uses a database specifically for caching to improve speed and efficiency.

- Database Tool: Redis (An in-memory, key-value data structure store).
- Role: Used as a temporary database to store frequently accessed weather data.
- Data Structure: Key-Value pairs where the Key is the city name (e.g., 'chennai') and the Value is the structured JSON data of the weather.
- TTL (Time-to-Live): Each data entry is set to automatically expire (e.g., after 1 hour). This ensures the cache always holds relatively fresh data and prevents stale information from being served.

3. Data Flow with Database

The Redis database's primary function is to optimize the data retrieval process:

1. Cache Check: The server checks the Redis database first for the requested city.
2. Cache Hit (Fastest): If data is present, it is served immediately from Redis, bypassing the external API call.
3. Cache Miss (Update): If data is not found, the server fetches data from the Open Weather API and then writes that data to the Redis database before sending it to the user.

Core Features Implementation

This phase involves integrating the core services (**Express**, **Axios**, **Redis**) to fulfill the main functionality: fetching weather data quickly and reliably.

1. REST API Endpoint (Express)

The backend exposes a single, primary endpoint that the frontend calls for weather data.

- **Endpoint:** GET /weather
- **Functionality:**
 - Receives the city name as a query parameter (e.g., /weather? city =London).
 - Performs basic input validation (checks if the city name is present).

2. Caching Service Integration (Redis)

This is the first logic executed to ensure fast response times and reduced API costs.

- **Cache Check:** The server attempts to retrieve the weather data from the **Redis** cache using the city name as the key.
- **Cache Hit:** If data is found, the server immediately returns the cached result to the frontend, labeled with source: "cache".

3. External API Integration (Axios & Open Weather)

If the data is not in the cache (Cache Miss), the server calls the external service.

- **API Call: Axios** is used to make an HTTP request to the **Open Weather API** using the user-provided city name and the private API key.
- **Error Handling (404):** The server specifically catches API errors, such as a **404 ("City not found")**, and returns a clean, user-friendly error response.

4. Data Processing & Cache Update

Upon receiving a successful response from Open Weather:

- **Data Transformation:** The server extracts only the necessary fields (city name, temperature, humidity, condition) from the large API response.
- **Cache Write:** The transformed data is stored in the **Redis** cache with a **Time-to-Live (TTL)** of 1 hour, making future requests for the same city faster.

5. Frontend Display

The frontend handles the final presentation of the data or error message.

- **Display Logic:** Based on the backend response, the frontend updates the UI to show the weather details and the **Source** (API or Cache).
- **Error Handling:** Displays clear, user-friendly error messages (e.g., "City not found," "Service unavailable") instead of technical errors.

Testing Core Features

Thorough testing is essential to verify the full integration between the Frontend, Backend, Redis Cache, and the External API.

1. API Functionality Test (Postman/c URL)

The goal is to ensure the **GET /weather** endpoint works correctly and handles basic city input.

- **Test Case 1: Valid City Search**
 - **Action:** Use Postman to send a request: GET /weather? city=Chennai.
 - **Expected Result:** Receive a successful **200 OK** response with structured weather data (temperature, humidity, condition) and the **source** field showing "api".
- **Test Case 2: Invalid City Search**
 - **Action:** Send a request with a clearly non-existent city: GET /weather? city=Invalid City Name.
 - **Expected Result:** Receive a **404 Not Found** response with the error message: "City not found".
- **Test Case 3: Missing Parameter**
 - **Action:** Send a request without the city query: GET /weather.
 - **Expected Result:** Receive a **400 Bad Request** response with the error message: "City is required".

2. Caching Test (Performance & Logic)

The goal is to verify that the **Redis** cache correctly stores and serves data, ensuring performance gains.

- **Test Case 4: Cache Miss (First Request)**
 - **Action:** Search for a new city (e.g., **Mumbai**) that has *never* been searched before.
 - **Verification:** Confirm the response has **source: "api"** and check server logs to verify a call was made to **Open Weather**.
- **Test Case 5: Cache Hit (Immediate Repeat)**
 - **Action:** Immediately search for the exact same city (Mumbai) again.
 - **Verification:** Confirm the response has **source: "cache"** and the response time is significantly faster (usually under 50ms). Server logs should show **no external API call**.
- **Test Case 6: Cache Expiration (TTL)**
 - **Action:** Search a city, wait for the **Time-to-Live (TTL)** period (e.g., 1 hour) to pass, and then search again.
 - **Verification:** The first search after the expiry should result in **source: "api"** (Cache Miss) again, confirming the data was correctly cleared.

3. Frontend & Display Test

The goal is to ensure the UI correctly displays the results and handles error states gracefully.

- **Test Case 7: Successful Display**
 - **Action:** Enter a valid city in the browser input and click "Get Weather".
 - **Verification:** The result card should correctly display the **City Name**, **Temperature**, **Humidity**, **Condition**, and the **Source** (API or Cache) without any UI glitches.
- **Test Case 8: Error Display**
 - **Action:** Enter an invalid city in the browser input.
 - **Verification:** The result area should immediately clear and display the user-friendly error message, such as "**City not found**", in red text

Version Control (GitHub)

Maintain project integrity and allow for collaboration.

Repository: